CSCI 264 — Assembly Language

Lab 8

Max

In this lab you will write a function, `max`, and call it three times from `main` to find the largest of four numbers. Here is the program in C:

```
long max(long x, long y) {
    if (x > y) {
        return x;
    }
    return y;
}

int main(void) {
    long a = 12, b = 31, c = -7, d = 25;
    long m1 = max(a, b);
    long m2 = max(c, d);
    return max(m1, m2);
}
```

Setup

Work in the ASM Visualizer's **Function Mode**, which we used in lecture. In Function Mode nothing is hidden. You write every function yourself, including `main`, and the program starts at `main`. There are no Initial Stack Values boxes, so `main` sets its own values with `mov`. The lines that start with a dot (`.text`, `.globl`, `.type`, `.size`) tell the assembler where each function starts and ends. Leave them alone.

Part 1: Write max

Open the starting code in the ASM Visualizer. It has an empty `max` and a `main` that calls it once:

```
    .text
    .globl  max
    .type   max, @function
max:
    # Part 1: write max here
    .size   max, .-max
    .globl  main
    .type   main, @function
main:
    push %rbp
    mov %rsp, %rbp
    mov $3, %rdi            # x = 3
    mov $8, %rsi            # y = 8
    call max                # %rax = max(3, 8)
    pop %rbp
    ret
    .size   main, .-main
```

Write `max`:

- `x` arrives in `%rdi` and `y` in `%rsi`. Leave the larger one in `%rax`.
- Start with `push %rbp` and `mov %rsp, %rbp`. End with `pop %rbp` and `ret`.
- The numbers can be negative. Use the signed jumps.
- Include comments to explain the code.

To test, change the two `mov` lines in `main`. Run all of these tests.

x	y	%rax
3	8	8
−5	−2	-2
−1	1	1
4	4	4

Part 2: The Largest of Four

Modify `main` find the max of four numbers, like so:

```
main:
  push %rbp
  mov %rsp, %rbp
  sub $48, %rsp           # room for a, b, c, d, and two more
  movq $12, -8(%rbp)      # a = 12
  movq $31, -16(%rbp)     # b = 31
  movq $-7, -24(%rbp)     # c = -7
  movq $25, -32(%rbp)     # d = 25

  # Part 2: your code here

  leave
  ret
  .size   main, .-main
```

Where it says *your code here*, find the largest of the four with three calls to `max`: `max(a, b)`, then `max(c, d)`, then `max` of those two results. Leave the answer in `%rax`.

To test, change the four `mov` lines. Run all of these tests. The first one gives the answer you should get. For the rest, work out the answer *before* you run it.

a	b	c	d	%rax
12	31	−7	25	31
5	1	9	2	
−4	−8	−1	−3	
7	7	7	7	
1	2	3	40	

Part 3: Short Answer

Answer each question in one or two sentences.

1. Where did you save `max(a, b)` before the second call? What would go wrong if you left it in `%rax`?
2. Each time you call `max` what is the value in `%rbp`. Why?
3. All three calls ran the same `ret` instruction, but each one went back to a different place. How did `ret` know where to go?

Submission

Keep your work in a text file called `lab8.txt`, set up like this:

```
Name:
Helped by:
Generative AI use:

Part 1
Tests: (one line per test: the values, what you expected,
        and what the visualizer gave)

Part 2
(your whole program: max and main, with comments)
Link:

Part 3
(short answers)
```

For the *Link*, paste the share link for your Part 2 program.

Submit `lab8.txt` on Canvas, there is no demo component for this lab. If a classmate helped you, put their name in the file. Disclose any use of Generative AI at the top.